University of Coimbra

Master in Computer Engineering

Assignment 2
Scenario Implementation

Communications and Cybersecurity



Students:
Gonçalo Nogueira (2022213646)
David Cameijo (2021246865)

Class: PL2
Academic Year: 2025/2026
Submission Date: 14 December 2025

Contents

1 Introduction **3**

1.1 Assignment Context 3

1.2 Implementation Plan and Architecture 3

1.2.1 (a) Final Plan to Instantiate Services 3

1.2.2 (b) Scenario Setup Diagram 5

2 Service Implementation **9**

2.1 Public Key Infrastructure (PKI) 9

2.1.1 Architecture Overview 9

2.1.2 Implementation 9

2.1.3 Issued Certificates 10

2.1.4 Revocation System 10

2.2 Public Web Service (web01) 10

2.2.1 Implementation Overview 10

2.2.2 Security Features 11

2.2.3 Security Policies Implementation 11

2.2.4 API Endpoints 12

2.2.5 Implementation Decisions 12

2.3 Database Service (db01) 13

2.3.1 Implementation Overview 13

2.3.2 Security Features 14

2.3.3 Security Policies Implementation 14

2.3.4 Operational Commands (“API”) 15

2.3.5 Implementation Decisions 15

2.4 SSH Service 16

2.4.1 Implementation Overview 16

2.4.2 Security Features 16

2.4.3 Security Policies Implementation 16

2.4.4 Operational Commands ("API") 17

2.4.5 Implementation Decisions 17

2.5 Firewall Service (fw01) 17

2.5.1 Overview 17

2.5.2 Security Features 17
2.5.3 Policy Summary 18
2.5.4 Interfaces and IPs 18
2.5.5 Allowed Flows 18
2.5.6 Operations 18
2.5.7 Logging 18
2.6 Test Evidence 18
2.6.1 Web Server Security Tests 18
2.6.2 Firewall Tests 24
2.6.3 Database Tests 26
2.6.4 SSH Certificate Authentication 27
2.6.5 TLS/HTTPS Encryption 29
2.6.6 PKI Integrity 30
2.6.7 Performance Measurement 31
2.6.8 Summary 32

Chapter 1

Introduction

This report documents our implementation of a secure infrastructure scenario for Assignment 2 of the Communications and Cybersecurity course. The work builds upon the architectural planning we developed in Assignment 1, translating the theoretical design into a functional, secure multi-service environment.

The implementation consists of five main components: a public web service, a database service, an SSH remote access service, a firewall service, and a PKI infrastructure. Each service enforces specific security policies and communicates securely with others, demonstrating practical application of cybersecurity principles learned throughout the course.

1.1 Assignment Context

Assignment 2 requires the implementation of a complete secure infrastructure scenario with multiple interconnected services. The assignment emphasizes the need for proper security policies, cryptographic mechanisms, and evidence-based testing to demonstrate that security measures function correctly.

Our implementation follows the design decisions from Assignment 1, including network segmentation (DMZ and internal network), use of virtualization technologies, and adherence to specified cryptographic standards (RSA 4096-bit for TLS, Ed25519 for SSH, SHA-256 for integrity).

1.2 Implementation Plan and Architecture

1.2.1 (a) Final Plan to Instantiate Services

Deployed Services

We implemented six services fulfilling Assignment 2 requirements:

Service	Technology	Purpose
PKI (ca01)	OpenSSL 3.0	Certificate issuance, CRL management
Web Server (web01)	Node.js + Express	Public web service with 2FA
Database (db01)	PostgreSQL 15	Data storage with mTLS
SSH Server (ssh01)	OpenSSH 9.x	Remote access with certificates
Firewall (fw01)	nftables	Traffic filtering, default DENY
Reverse Proxy (nginx01)	Nginx 1.24	HTTPS termination

Table 1.1: Implemented services and technologies

Deployment Steps

The deployment follows a dependency-driven sequence:

1. **Phase 1 - Network Setup**: Create DMZ (172.18.0.0/16) and Internal (172.19.0.0/16) networks with static IP assignments
2. **Phase 2 - PKI Initialization**: Generate Root CA → Intermediate CA → Server certificates → Client certificates → SSH CA
3. **Phase 3 - Service Deployment**:

```
docker-compose up -d db01 web01 fw01 nginx01
cd SSH01 && sudo ./setup-ssh-vm.sh
```

4. **Phase 4 - Verification**: Test connectivity, certificate validation, firewall rules, and configuration integrity

Changes from Assignment 1

Key modifications for Assignment 2:

1. **Network Segmentation**: Added DMZ (172.18.x) and Internal (172.19.x) networks with static IPs for firewall rules
2. **SSH Deployment**: Moved from Docker to Kali VM for realistic remote access testing
3. **CA Isolation**: Firewall blocks all ca01 traffic (internal only, certificate issuance completed pre-deployment)
4. **Auth & AuthZ**: Added JWT authentication, fail2ban (2 failures=15min), RBAC (admin/read/readwrite)

5. **Integrity**: SHA-256 checksums with automatic (fw01, web01) and manual (db01, ssh01, ca01) verification

6. **2FA**: TOTP-based with Google Authenticator

1.2.2 (b) Scenario Setup Diagram

This section provides diagrams illustrating service deployment and network architecture.

Architecture Diagram

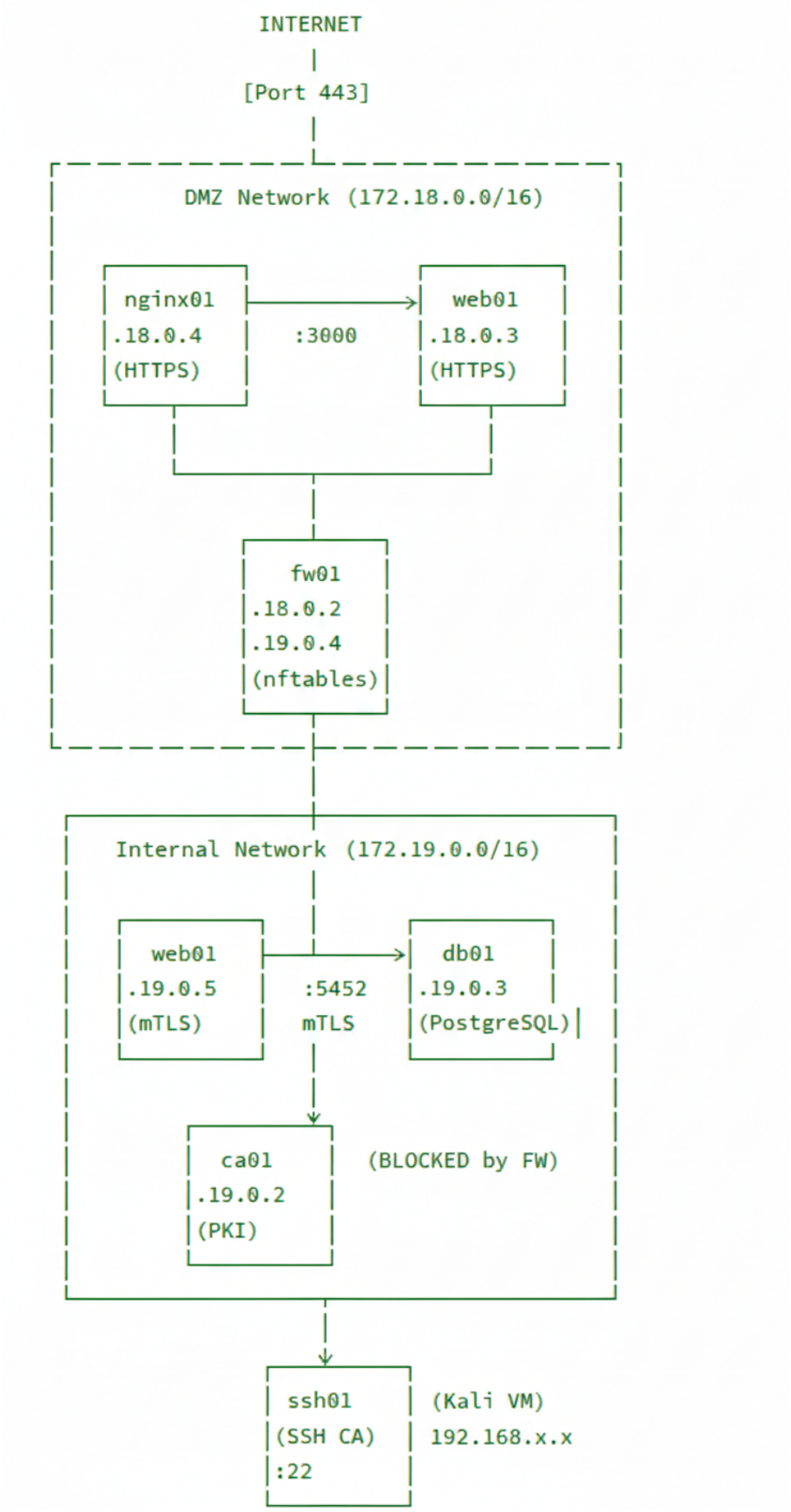


Figure 1.1: System Architecture Diagram

Legend:

- Solid lines (—) indicate allowed network connections
- Arrow direction (→) shows traffic flow
- Port numbers indicate service endpoints
- IP addresses show static assignments

Infrastructure Description

Network Segmentation The infrastructure implements a two-tier network segmentation model:

1. **DMZ Network (172.18.0.0/16)**
   - Purpose: Hosts publicly accessible services
   - Gateway: 172.18.0.1
   - Services: nginx01 (reverse proxy), web01 (dual-homed), fw01 (dual-homed)
   - External Access: Port 443 exposed to internet
   - Security: Firewall rules strictly control traffic flow
2. **Internal Network (172.19.0.0/16)**
   - Purpose: Hosts backend services and sensitive data
   - Gateway: 172.19.0.1
   - Services: db01 (database), ca01 (PKI), web01 (dual-homed), fw01 (dual-homed)
   - Isolation: No direct external access; CA completely blocked
   - Security: mTLS required for all connections

(c) Data Flows and Service Inventory

Flow	Protocol	Port	Encryption	Purpose
Client→nginx01	HTTPS	443	TLS 1.3	User web access
nginx01→web01	HTTPS	3000	TLS 1.3	Reverse proxy
web01→db01	PostgreSQL	5432	TLS + mTLS	Database queries
Admin→ssh01	SSH	22	Ed25519	Remote access
ANY→ca01	-	-	BLOCKED	CA isolation

Table 1.2: Data flows with ports, protocols, and encryption

Network Segmentation: DMZ (172.18.x): nginx01 (.4), web01 (.3), fw01 (.2); Internal (172.19.x): ca01 (.2), db01 (.3), fw01 (.4), web01 (.5). Firewall enforces default DENY with specific allow rules.

Virtualization Model

Container Orchestration: Docker 24.x+ with Docker Compose for service orchestration, custom bridge networks with static IPs, persistent volumes for database/certificates/logs.

Service	Base Image	Rationale
nginx01, web01, db01	Alpine Linux 3.x	Minimal (5MB), security-focused
fw01, ca01	Debian 12	Full-featured (nftables, OpenSSL)
ssh01	Kali Linux 2024.x	Pre-configured SSH server

Table 1.3: Operating systems and deployment rationale

Security Boundaries: Network isolation (custom Docker networks), filesystem isolation (minimal volume mounts, read-only where possible), process isolation (minimal privileges), certificate-based trust (no shared secrets).

Chapter 2

Service Implementation

2.1 Public Key Infrastructure (PKI)

2.1.1 Architecture Overview

Hierarchy: Two-tier PKI with Root CA (self-signed, offline) and Intermediate CA (signs all certificates). **Algorithms:** RSA 4096-bit keys, SHA-256 signatures. **Validity:** Root CA (10 years), Intermediate CA (5 years), server/client certificates (1 year).

2.1.2 Implementation

Directory Structure: `PKI/root-ca/` and `PKI/intermediate-ca/` with `private/` (600 permissions), `certs/`, `crl/`, `checksums/`, `scripts/`.

Automated Scripts:

- `init-ca.sh`: Initialize CA databases (index.txt, serial)
- `generate-root-ca.sh`: Create Root CA (RSA 4096, SHA-256, 10-year validity)
- `generate-intermediate-ca.sh`: Create Intermediate CA signed by Root CA
- `issue-server-cert.sh <hostname>`: Issue server certificates with `extendedKeyUsage = serverAuth`
- `issue-client-cert.sh <name>`: Issue client certificates with `extendedKeyUsage = clientAuth`
- `revoke-cert.sh <cert>`: Revoke certificates and update CRL
- `verify-integrity.sh`: Verify SHA-256 checksums of critical files

Security Features:

- Dedicated user `ca` with restricted permissions (private keys 600, directories 700)

- SHA-256 checksums for all critical files (keys, configs, database)
- CRL generation with 30-day validity (PEM and DER formats)
- Certificate chain files included for easy deployment

2.1.3 Issued Certificates

Certificate	Serial	Extended Key Usage
web01.org.local	1001	serverAuth (TLS server)
web02.org.local	1004	serverAuth (TLS server)
db01.org.local	1002	serverAuth (TLS server)
nginx01.org.local	1005	serverAuth (TLS server)
web02-client	1006	clientAuth (mTLS client)

Table 2.1: Issued certificates for infrastructure services

Certificate Properties: RSA 4096-bit, SHA-256 signature, 1-year validity, Subject CN and SAN with hostname, Key Usage (Digital Signature, Key Encipherment), certificate chain files included.

2.1.4 Revocation System

CRL Management: `revoke-cert.sh` updates database, `generate-crl.sh` creates CRL (30-day validity, PEM/DER formats), `verify-revocation.sh` checks revocation status.

Test Verification: Certificate `test-revoke.org.local` successfully revoked, `openssl verify -crl_check` returned `error 23: certificate revoked`, confirming functional revocation system (Assignment 2 PKI lifecycle requirement).

2.2 Public Web Service (web01)

2.2.1 Implementation Overview

Technology Stack: Node.js 18 (Alpine Linux), Express.js framework, deployed as Docker container with dual-homed networking (DMZ 172.18.0.3 + Internal 172.19.0.5).

2.2.2 Security Features

Feature	Implementation
2FA	TOTP (RFC 6238) with speakeasy, QR code generation, Google Authenticator compatible
JWT Authentication	HMAC-SHA256 signed tokens, 24-hour expiry, embedded role for RBAC
RBAC	Three roles: admin (read/write), readwrite (read/write), read (read only) enforced on /data endpoints
Fail2ban	In-memory tracking, 2 failed attempts $\Rightarrow$ $15 minban per username$
TLS (Server)	Port 3000, web01.org.local certificate (RSA 4096), TLS 1.3, receives from nginx
mTLS (Client)	Client certificate (web02-client) for db01 authentication, extendedKeyUsage = clientAuth
Password Hashing	bcrypt with cost=10 for database storage
Integrity	SHA-256 checksums via config-integrity.sh; startup logs integrity hash for .env

Table 2.2: Web service security features (Assignment 2 Table 1 compliance)

2.2.3 Security Policies Implementation

Two-Factor Authentication (2FA)

The service implements TOTP-based 2FA for user registration:

Registration Flow:

1. User submits username and password to /register endpoint
2. Password hashed with bcrypt (10 salt rounds)
3. TOTP secret generated using speakeasy library
4. QR code generated for secret (otpauth:// URI)
5. User data stored in PostgreSQL with encrypted connection
6. Response includes QR code for authenticator app setup

Implementation:

```
const secret = speakeasy.generateSecret({
  name: `SecureApp (${username})`,
  length: 32
```

```
});
const qrCode = await QRCode.toDataURL(secret.otpauth_url);
// Store secret.base32 in database
```

Traffic Encryption

All traffic to/from web01 is encrypted:

- **Client → Nginx**: HTTPS/TLS 1.3 (port 443)
- **Nginx → Web01**: HTTPS/TLS 1.3 (port 3000) with certificate verification
- **Web01 → Database**: PostgreSQL TLS with mTLS client authentication (port 5432)

Configuration File Integrity

extbfDecision: Enforce configuration integrity for server.js and package.json.

extbfMethod: SHA-256 baseline stored in /app/config-checksums.sha256 and verified by config-integrity.sh at container start (evidence: checksum pass). .env integrity hash is logged with HMAC-SHA256.

extbfStatus: Implemented and verified.

2.2.4 API Endpoints

extbfEndpoint	**Method**	**Description / Auth**
/health	GET	Health check (no auth)
/register	POST	User registration + TOTP secret/QR (rate-limited)
/login	POST	Username/password + TOTP (if enabled); issues JWT with role
/data	GET	List data (auth required; all roles read)
/data	POST	Create data (auth; roles readwrite/admin)
/data/:id	PUT	Update data (auth; roles readwrite/admin)

Table 2.3: Web Service API Endpoints and authorization

2.2.5 Implementation Decisions

Dual Network Connectivity

Decision: web01 connected to both DMZ and internal networks.

Justification: Receives public traffic from nginx (DMZ) while accessing database (internal network). Firewall rules restrict traffic flows: only nginx→web01:3000 and web01→db01:5432 allowed.

Client Certificate Authentication

Decision: Use mTLS with client certificates for database connections.

Justification: Provides stronger authentication than password-only. Certificate must have extendedKeyUsage = clientAuth. PostgreSQL configured with clientcert=verify-ca in pg_hba.conf.

Static IP Assignment

Decision: Assign static IPs to web01 (172.18.0.3 in DMZ, 172.19.0.5 in internal).

Justification: Firewall rules reference specific IP addresses. Dynamic IPs would break firewall rules on container restart. Docker IPAM used to ensure IP persistence.

2.3 Database Service (db01)

2.3.1 Implementation Overview

Technology Stack: PostgreSQL 15 (official image) running in Docker, single-homed on the internal network (172.19.0.3) only.

2.3.2 Security Features

Feature	Implementation
TLS (Server)	db01.org.local certificate (RSA 4096) served on 5432; ssl=on with CA chain
mTLS (Client auth)	pg_hba.conf enforces hostssl + clientcert=verify-ca; clients must have EKU=clientAuth
Authentication	SCRAM-SHA-256 passwords + client certificates (web01 uses web01-client cert)
RBAC	Roles via init-users.sql (admin + app users); API layer enforces read/readwrite/admin
Network Isolation	Internal network only; firewall allows web01→db01:5432; no DMZ access
Integrity	config-integrity.sh verifies SHA-256 for postgresql.conf, pg_hba.conf, init-users.sql
TLS Key Handling	Entry point installs certs to /var/lib/postgresql/tls-fixed with strict 600/644 perms
Logging	TLS/mTLS connection evidence in web01 logs; db01 via Docker logging

Table 2.4: Database security features (Assignment 2 Table 1 compliance)

2.3.3 Security Policies Implementation

TLS and mTLS Enforcement

Server TLS: ssl=on with server.crt/server.key and ca.crt installed under /var/lib/postgresql/tls-fixed (permissions tightened by postgres-entrypoint.sh).

Client Certificate Verification: postgres-entrypoint.sh prepends strict rules:

```
hostssl all all 0.0.0.0/0 scram-sha-256 clientcert=verify-ca
hostssl all all ::0/0    scram-sha-256 clientcert=verify-ca
```

Legacy permissive host lines are commented to prevent non-TLS paths.

Authentication and Roles

- **Passwords:** SCRAM-SHA-256 (PostgreSQL 15 default).
- **Certificates:** Clients must present certs signed by the Intermediate CA with EKU=clientAuth; web01 uses web01-client.crt,key.
- **Roles:** init-users.sql provisions admin and app users; the API (JWT roles) gates read vs readwrite/admin on `/data`.

Configuration Integrity

Decision: Enforce integrity for database configuration and bootstrap SQL.

Method: /usr/local/bin/config-integrity.sh verify checks SHA-256 for postgresql.conf, pg_hba.conf, and init-users.sql; rebaseline with `generate` only after approved changes.

Status: Implemented and verified (evidence shows checksum pass).

Network Isolation

- Single-homed on internal network (172.19.0.3); not attached to DMZ.
- Firewall allows only web01 (172.19.0.5) to reach db01:5432.
- No direct public ingress; access is mediated through web01.

2.3.4 Operational Commands ("API")

- Verify mTLS rule: `docker exec db01 grep "clientcert" /var/lib/postgresql/data/pg_hb`
- Check app-side evidence: `docker logs web01 | grep -i postgres`
- Integrity check: `docker exec db01 /usr/local/bin/config-integrity.sh verify`
- Reload config: `docker exec db01 psql -c "SELECT pg_reload_conf();"`

2.3.5 Implementation Decisions

mTLS Client Certificates

Decision: Require clientAuth EKU certs for all remote DB access.

Justification: Prevents password-only access and enforces device identity; serverAuth-only certs are rejected for client auth.

Internal Network Only

Decision: Keep db01 off the DMZ.

Justification: Limits exposure; only web01 can reach db01 through firewall allowlist.

Strict pg_hba Ordering

Decision: Prepend hostssl+clientcert rules and comment permissive host lines in postgres-entrypoint.sh.

Justification: Ensures no plaintext or non-cert auth paths remain after container start.

2.4 SSH Service

2.4.1 Implementation Overview

Technology Stack: OpenSSH server configured on the VM with certificate-based user authentication, aligned to an internal-only access policy.

2.4.2 Security Features

Feature	Implementation
Certificate Auth	User keys signed by SSH CA; server trusts CA public key (`ca.pub`)
Passwordless Login	`admin-key` + `admin-key-cert.pub` used; passwords disabled for admin
Policy Enforcement	Login requires valid cert; attempts without cert fail with *Permission denied (publickey)*
Session Limits	`MaxSessions 5` in `/etc/ssh/sshd_config`
Integrity	`verify-ssh-integrity.sh` validates checksums for `sshd_config` and `ca.pub`
Firewall	Only internal admin source permitted; DMZ access blocked by firewall policy

Table 2.5: SSH security features (Assignment 2 Table 1 compliance)

2.4.3 Security Policies Implementation

Certificate Authentication

Server Trust: `TrustedUserCAKeys` points to the SSH CA public key; user keys must be signed.

User Certs: `admin-key` signed to `admin-key-cert.pub`; login succeeds without password when valid cert presented.

Negative Test: Without cert, login fails: *Permission denied (publickey)* (evidence figure included).

Configuration Integrity

Decision: Enforce integrity verification of SSH server configuration and CA key.

Method: `verify-ssh-integrity.sh` checks SHA-256 checksums; evidence shows pass.

2.4.4 Operational Commands ("API")

- Certificate login: `ssh -i PKI/ssh-ca/user-certs/admin-key admin@<host>`
- Negative test: `ssh admin@<host>` ⇒ *Permission denied (publickey)*
- Integrity check: `sudo verify-ssh-integrity.sh`
- Config inspection: `grep MaxSessions /etc/ssh/sshd_config`

2.4.5 Implementation Decisions

Enforce Cert-only Auth

Decision: Disable password login for admin; require signed user certificates.

Justification: Reduces credential attack surface; relies on cryptographic identity issued by SSH CA.

Session Limits

Decision: Configure `MaxSessions 5` to cap concurrent sessions.

Justification: Prevents resource exhaustion via excessive session multiplexing.

Firewall Restriction

Decision: Restrict SSH ingress to internal/admin sources; block DMZ.

Justification: Aligns with least-privilege network access and reduces exposure.

2.5 Firewall Service (fw01)

2.5.1 Overview

Stack: nftables with default **DROP** and a minimal allowlist between DMZ and internal.

2.5.2 Security Features

Feature	Implementation
Default DENY	DROP base policy; only required flows allowed
Allows	22 (admin), 53 (DNS), 443 (HTTPS), 3000 (web01), 5432 (PostgreSQL)
CA Isolation	Block 172.19.0.2 with log prefix `[FW-DENY-CA-ACCESS]`
Integrity	`integrity-check.sh` verifies `nftables.conf` and `entrypoint.sh`

Table 2.6: Firewall features (concise)

2.5.3 Policy Summary

- Default **DROP**; allow client→nginx:443, nginx→web01:3000, web01→db01:5432, SSH:22. - CA (172.19.0.2) explicitly blocked and logged.

2.5.4 Interfaces and IPs

- DMZ: fw01=172.18.0.2, nginx=172.18.0.4, web01=172.18.0.3 - Internal: fw01=172.19.0.4, db01=172.19.0.3, ca01=172.19.0.2, web01=172.19.0.5 - Note: Static IPs pinned via docker-compose IPAM to keep rules stable.

2.5.5 Allowed Flows

- nginx→web01:3000 (internal HTTPS proxy path) - web01→db01:5432 (PostgreSQL with mTLS) - Internal→DNS:53 (UDP/TCP) - ANY→fw01:22 (admin SSH) - External HTTPS terminates at nginx:443 (DMZ), then proxied inside.

2.5.6 Operations

- List rules: `docker exec fw01 nft list ruleset` - Verify CA block: `docker exec fw01 nft list ruleset | grep "172.19.0.2"` - Integrity: `docker exec fw01 /usr/local/bin/int verify`

2.5.7 Logging

- Drops logged with prefixes (e.g., `[FW-DENY-CA-ACCESS]`). Rule counters visible via `nft list ruleset -a`.

2.6 Test Evidence

This section provides visual evidence of policy implementation and testing for all services, demonstrating compliance with Assignment 2 Table 1 requirements.

2.6.1 Web Server Security Tests

2FA Registration and QR Code Generation

Test: User registration with TOTP-based two-factor authentication.

Command:

```
curl -k -X POST https://localhost/register \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser","password":"TestPass123"}'
```



Figure 2.1: 2FA registration response with QR code and secret

Result: SUCCESS - User registered with TOTP secret generated, QR code provided for Google Authenticator setup.

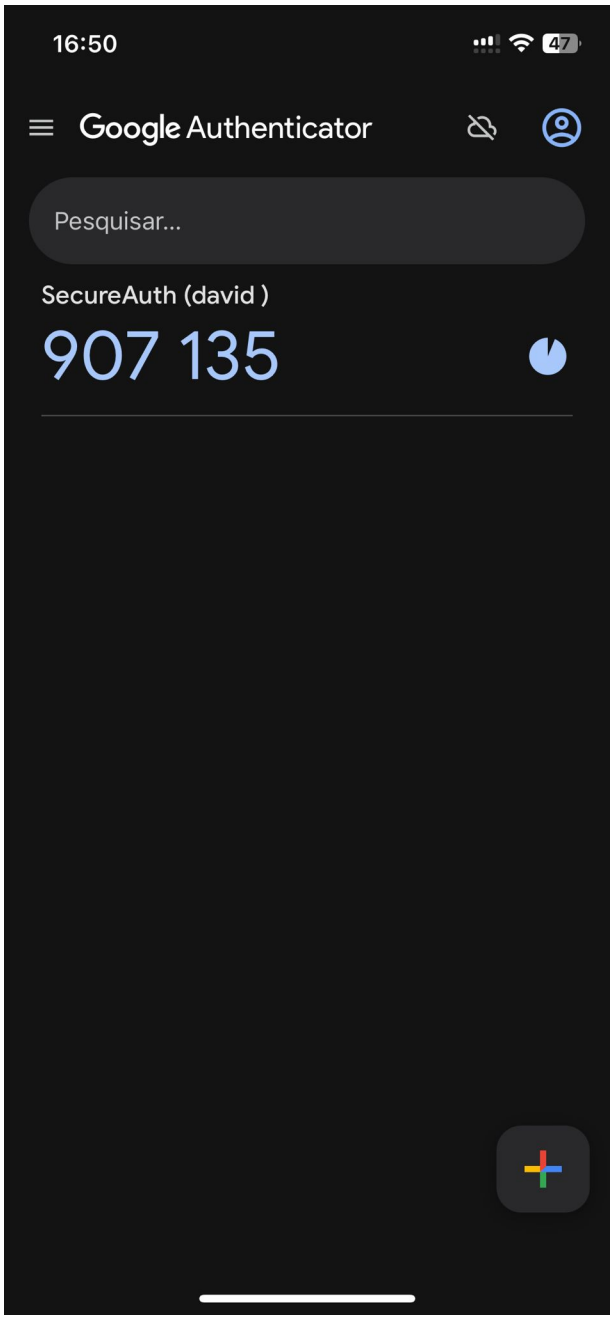


Figure 2.2: QR code scanned in Google Authenticator app

Policy Verified: Web Server 2FA requirement (Assignment 2 Table 1).

JWT Authentication

Test 1: Accessing protected endpoint without authentication token.

```
curl -k -X GET https://localhost/data
```

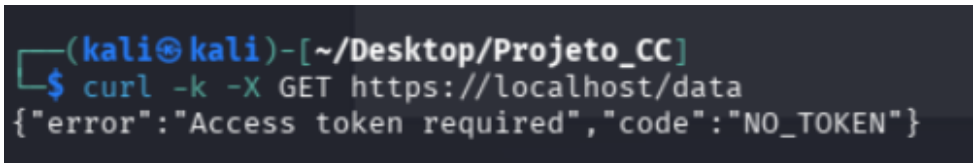


Figure 2.3: Access denied without JWT token (401 Unauthorized)

Result: SUCCESS - Request rejected with 401 error "Access token required".

Test 2: Login and receive JWT token.

```
curl -k -X POST https://localhost/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"adminpassword"}'
```

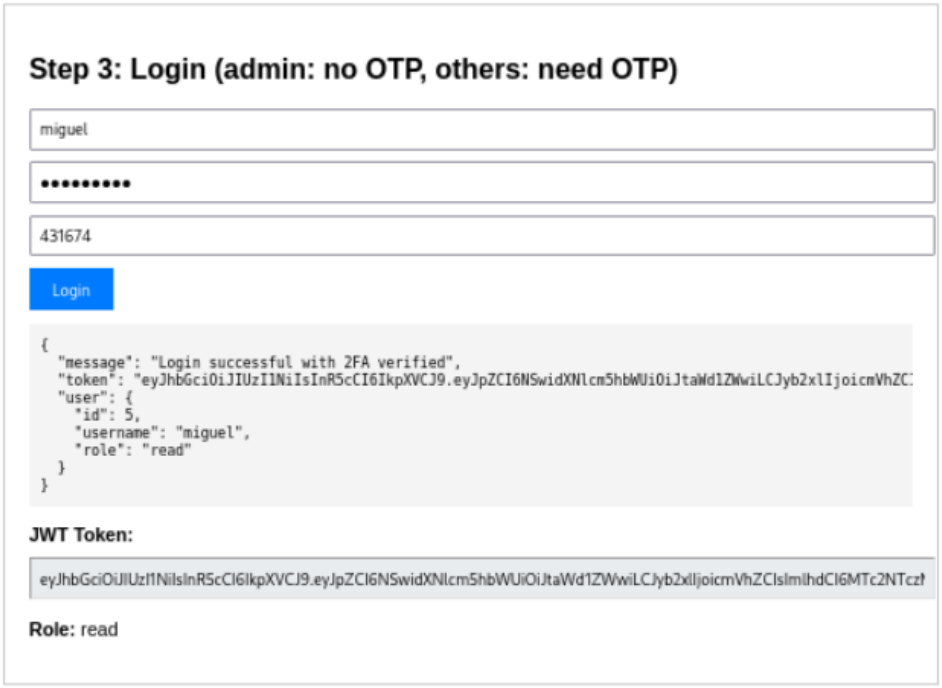


Figure 2.4: Successful login with JWT token and role information

Result: SUCCESS - JWT token received with embedded role (admin), 24-hour expiry.

Policy Verified: Authentication mechanism for API access.

RBAC Authorization - Read-Only User

Test: User with role='read' attempts to create data (write operation).

```
curl -k -X POST https://localhost/data \
  -H "Authorization: Bearer <user-token>" \
```

```
  -H "Content-Type: application/json" \
  -d '{"title":"Test","content":"Should fail"}'
```

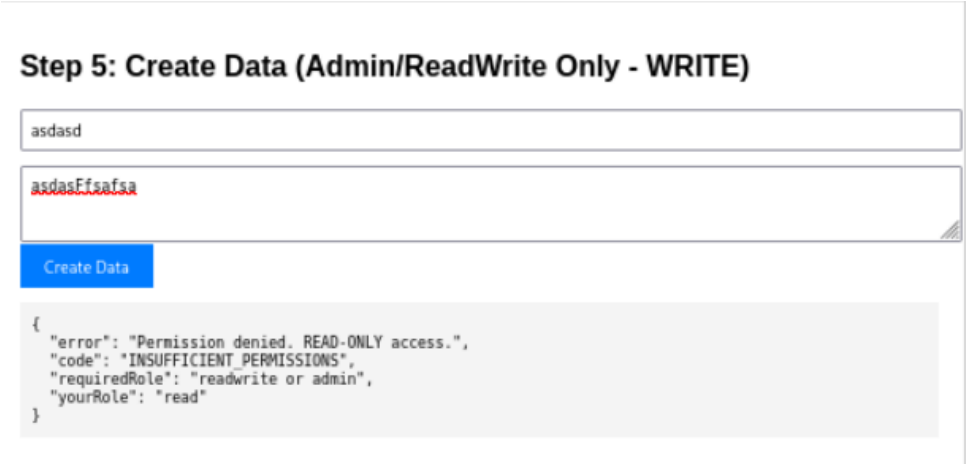


Figure 2.5: Read-only user denied write access (403 Forbidden)

Result: SUCCESS - Request denied with 403 error "Permission denied. READ-ONLY access."

Policy Verified: Database RBAC requirement - different users have distinct access (Assignment 2 Table 1).

RBAC Authorization - Admin User

Test: Admin user creates data (write operation).

```
curl -k -X POST https://localhost/data \
  -H "Authorization: Bearer <admin-token>" \
  -H "Content-Type: application/json" \
  -d '{"title":"Admin Post","content":"Test data"}'
```



Figure 2.6: Admin user successfully creates data (200 OK)

Result: SUCCESS - Data created successfully, response includes new record with id.

Policy Verified: Admin role has full read/write access to data.

Fail2ban Protection

Test: Multiple failed login attempts to trigger account lockout.

```
# Attempt 1
curl -k -X POST https://localhost/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"wrong"}'
# Response: {"error":"Invalid credentials","remainingAttempts":2}

# Attempt 2
curl -k -X POST https://localhost/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"wrong"}'
# Response: {"error":"Invalid credentials","remainingAttempts":1}

# Attempt 3
curl -k -X POST https://localhost/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"wrong"}'
# Response: {"error":"Account locked after 2 failed attempts..."}
```



Figure 2.7: Failed login attempts with remaining attempts counter



Figure 2.8: Account locked after exceeding maximum failed attempts (403)

Result: SUCCESS - Account locked for 15 minutes after 2 failed attempts. Correct password also blocked during ban period.

Policy Verified: SIEM fail2ban requirement - "If login fails more than 2 times, ban the user" (Assignment 2 Table 1).

Configuration Integrity Verification

Test: Verify SHA-256 checksums of configuration files.

```
docker exec web01 /usr/local/bin/config-integrity.sh verify
```



Figure 2.9: Configuration integrity verification for web01

Result: SUCCESS - All checksums matched baseline (server.js, package.json).

Policy Verified: Web Server integrity requirement (Assignment 2 Table 1).

2.6.2 Firewall Tests

Default DENY Policy

Test: Verify firewall rules with default DROP policy.

```
docker exec fw01 nft list ruleset
```

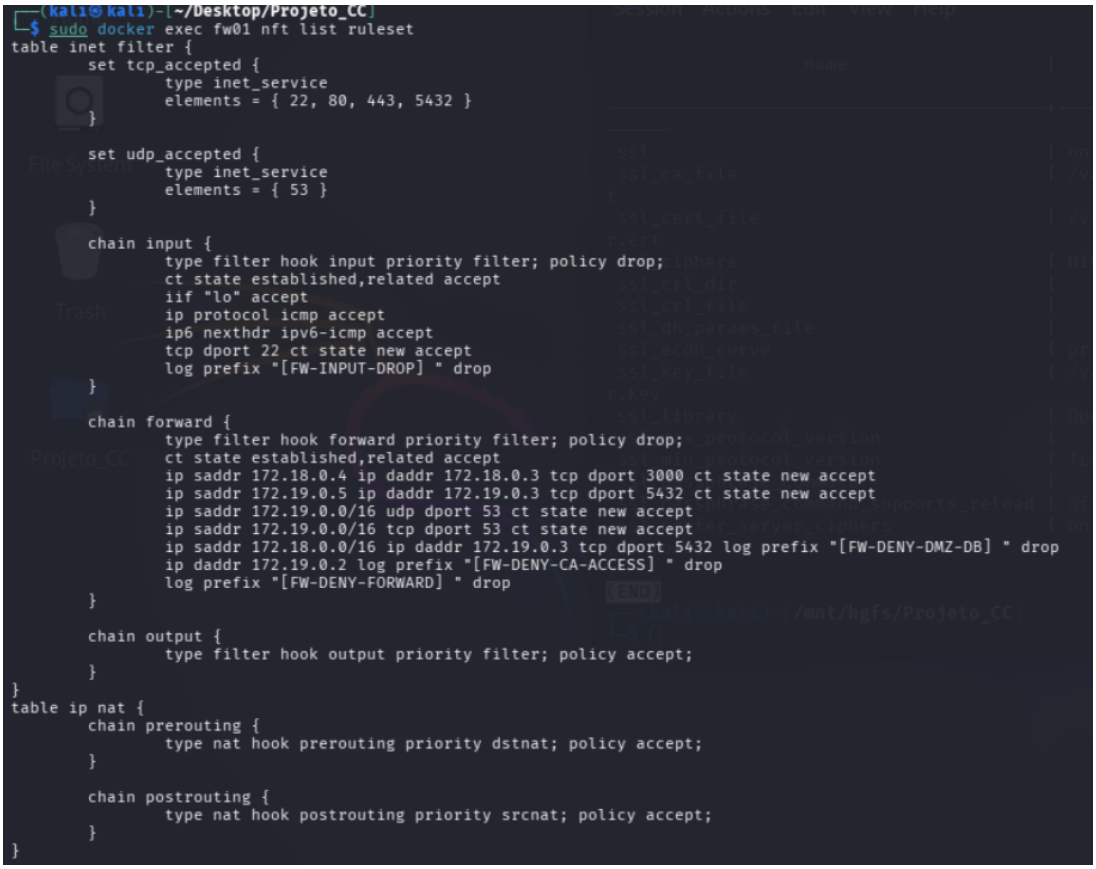


Figure 2.10: nftables ruleset showing default DROP policy and specific allow rules

Result: SUCCESS - Default policy is DROP, specific allow rules for ports 22, 53, 443, 3000, 5432.

Policy Verified: Firewall default DENY requirement (Assignment 2 Table 1).

CA Isolation

Test: Verify CA (172.19.0.2) is blocked by firewall.

```
docker exec fw01 nft list ruleset | grep "172.19.0.2"
```

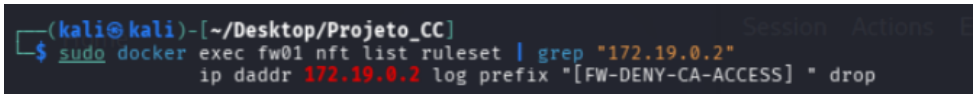


Figure 2.11: Firewall rule blocking all traffic to CA (172.19.0.2)

Result: SUCCESS - All traffic to 172.19.0.2 explicitly dropped with logging prefix "[FW-DENY-CA-ACCESS]".

Policy Verified: CA secure access control (Assignment 2 Table 1).

Firewall Integrity

Test: Verify firewall configuration integrity.

```
docker exec fw01 /usr/local/bin/integrity-check.sh verify
```

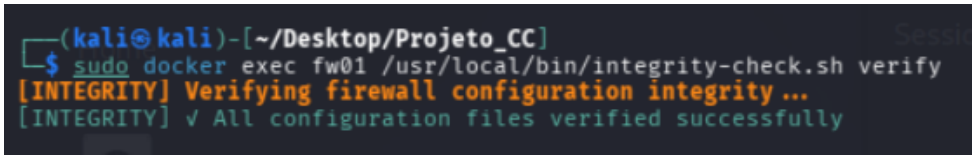


Figure 2.12: Firewall configuration integrity verification

Result: SUCCESS - Checksums verified for nftables.conf and entrypoint.sh.
Policy Verified: Firewall integrity requirement (Assignment 2 Table 1).

2.6.3 Database Tests

TLS Encryption

Test: Verify PostgreSQL TLS connection from web01.

```
docker logs web01 | grep -i "postgres\|database\|connected"
```

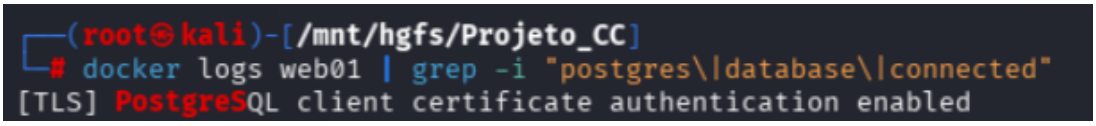


Figure 2.13: Web01 logs showing successful TLS connection to database

Result: SUCCESS - Connection established with TLS encryption, client certificate presented.
Policy Verified: Database encryption requirement (Assignment 2 Table 1).

mTLS Client Authentication

Test: Verify pg_hba.conf requires client certificate verification.

```
docker exec db01 grep "clientcert" /var/lib/postgresql/data/pg_hba.conf
```

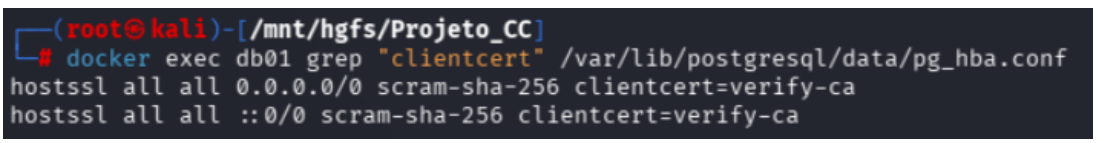


Figure 2.14: pg_hba.conf showing clientcert=verify-ca requirement

Expected output: `hostssl all all 0.0.0.0/0 scram-sha-256 clientcert=verify-ca`

Result: SUCCESS - PostgreSQL configured to require and verify client certificates for all SSL connections.

Additional verification: Check web01 logs for successful database connection with client certificate:

```
docker logs web01 2>&1 | grep -i "database\|connected\|postgres"
```

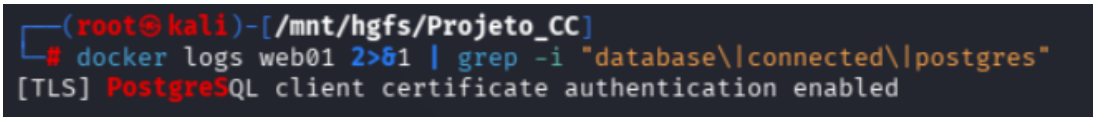


Figure 2.15: Web01 logs showing successful mTLS connection to database

Policy Verified: Database mTLS authentication with client certificate verification (Assignment 2 Table 1).

Database Integrity

Test: Verify database configuration checksums.

```
docker compose exec db01 /usr/local/bin/config-integrity.sh verify
```

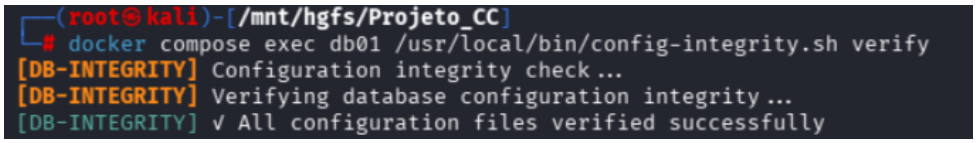


Figure 2.16: Database configuration integrity verification

Result: SUCCESS - Checksums matched for postgresql.conf, pg_hba.conf, init-users.sql.

Policy Verified: Database integrity requirement (Assignment 2 Table 1).

2.6.4 SSH Certificate Authentication

SSH Connection Test

Test: Connect to SSH server using certificate authentication.

```
ssh -i PKI/ssh-ca/user-certs/admin-key admin@192.168.1.100
```

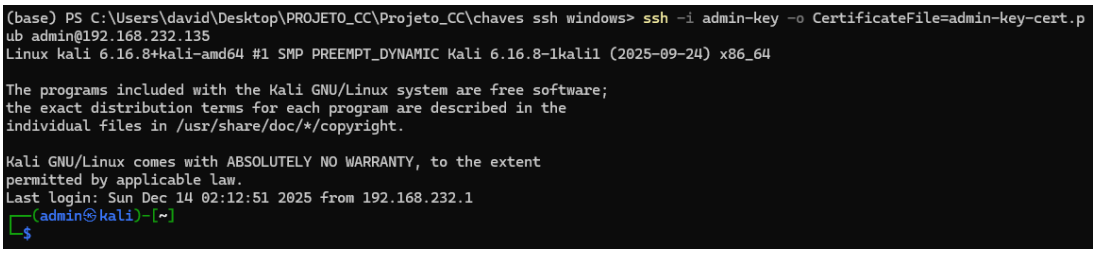


Figure 2.17: Successful SSH connection using certificate (no password prompt)

Result: SUCCESS - Connected without password prompt, certificate authentication successful.

Policy Verified: SSH certificate authentication requirement (Assignment 2 Table 1).

SSH Certificate Enforcement (No Cert)

extbfTest: Attempt SSH login without presenting a signed client certificate.

```
ssh admin@192.168.232.135
# Fails with: Permission denied (publickey).
```

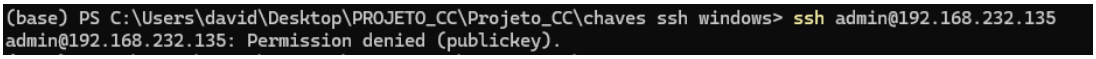


Figure 2.18: SSH login without client certificate is rejected (Permission denied, publickey)

extbfResult: SUCCESS - Access denied when no client certificate is provided.

extbfPolicy Verified: SSH certificate authentication enforced (Assignment 2 Table 1).

SSH Session Limit

Test: Verify MaxSessions configuration.

```
grep MaxSessions /etc/ssh/sshd_config
```

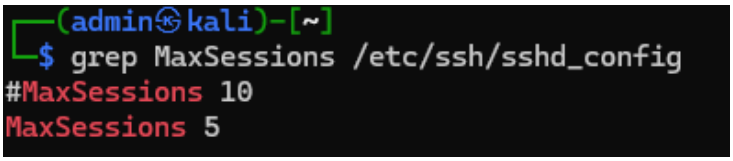


Figure 2.19: SSH MaxSessions set to 5

Result: SUCCESS - MaxSessions 5 configured in sshd_config.

Policy Verified: SSH session limit requirement (Assignment 2 Table 1).

SSH Integrity

Test: Verify SSH configuration integrity.

```
sudo verify-ssh-integrity.sh
```

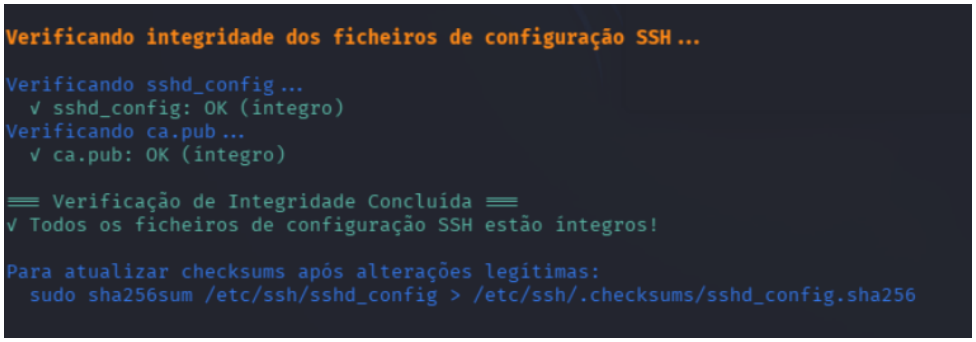


Figure 2.20: SSH configuration integrity verification (sshd_config, ca.pub)

Result: SUCCESS - All checksums verified for sshd_config and ca.pub.
Policy Verified: SSH integrity requirement (Assignment 2 Table 1).

2.6.5 TLS/HTTPS Encryption

TLS Version Verification

Test: Verify TLS 1.3 is used for HTTPS connections.

```
curl -k -v https://localhost/health 2>&1 | grep TLSv1.3
```

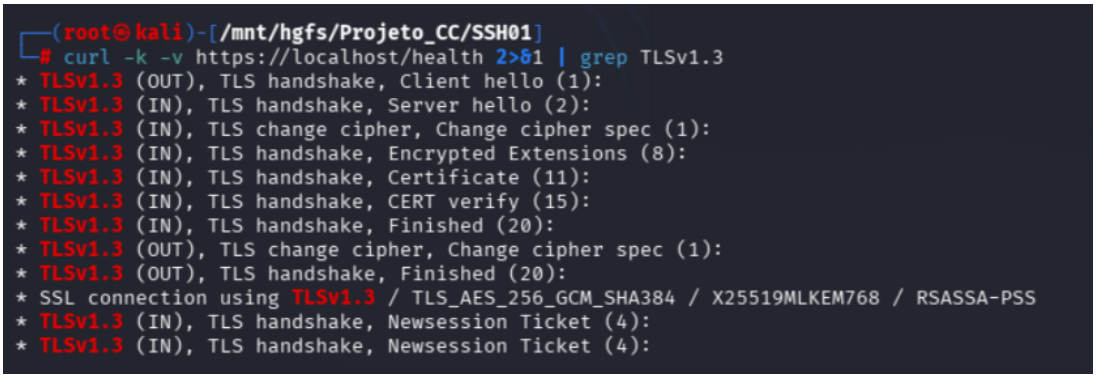


Figure 2.21: TLS 1.3 connection established

Result: SUCCESS - TLS 1.3 protocol used, strong cipher suite (TLS_AES_256_GCM_SHA384).
Policy Verified: Encrypted traffic requirements (all services).

Certificate Verification

extbfTest: Verify HTTPS certificate chain using the deployed CA bundle.

```
openssl s_client -connect localhost:443 -showcerts \
  -CAfile /mnt/hgfs/Projeto_CC/nginx/certs/ca-chain.crt </dev/null 2>&1 | \
  grep -E "(subject|issuer|CN=)"
```

```
┌──(root㉿kali)-[/mnt/hgfs/Projeto_CC]
└─# openssl s_client -connect localhost:443 -showcerts \
  -CAfile /mnt/hgfs/Projeto_CC/nginx/certs/ca-chain.crt </dev/null 2>&1 | \
  grep -E "(subject|issuer|CN=)"
depth=2 C=PT, ST=Portugal, L=Coimbra, O=Assignment 2 PKI, OU=Root CA, CN=Root CA - Assignment 2
depth=1 C=PT, ST=Portugal, L=Coimbra, O=Assignment 2 PKI, OU=Intermediate CA, CN=Intermediate CA - Assignment 2
depth=0 CN=web01.org.local
 0 s:CN=web01.org.local
   i:C=PT, ST=Portugal, L=Coimbra, O=Assignment 2 PKI, OU=Intermediate CA, CN=Intermediate CA - Assignment 2
 1 s:C=PT, ST=Portugal, L=Coimbra, O=Assignment 2 PKI, OU=Intermediate CA, CN=Intermediate CA - Assignment 2
   i:C=PT, ST=Portugal, L=Coimbra, O=Assignment 2 PKI, OU=Root CA, CN=Root CA - Assignment 2
subject=CN=web01.org.local
issuer=C=PT, ST=Portugal, L=Coimbra, O=Assignment 2 PKI, OU=Intermediate CA, CN=Intermediate CA - Assignment 2
```

Figure 2.22: Certificate chain verification (Root CA → Intermediate CA → web01.org.local)

extbfResult: SUCCESS - Chain validates with ca-chain.crt; leaf CN=web01.org.local signed by Intermediate CA, verified up to Root CA.

extbfPolicy Verified: PKI certificate infrastructure.

2.6.6 PKI Integrity

extbfTest: Verify PKI critical assets (root/intermediate keys, certs, CRL, index/serial files) against stored SHA-256 checksums.

```
cd PKI
./scripts/verify-integrity.sh
```

extbfResult: SUCCESS - All PKI files match the recorded checksums; integrity verified.

extbfPolicy Verified: PKI integrity protection (Assignment 2 Table 1).

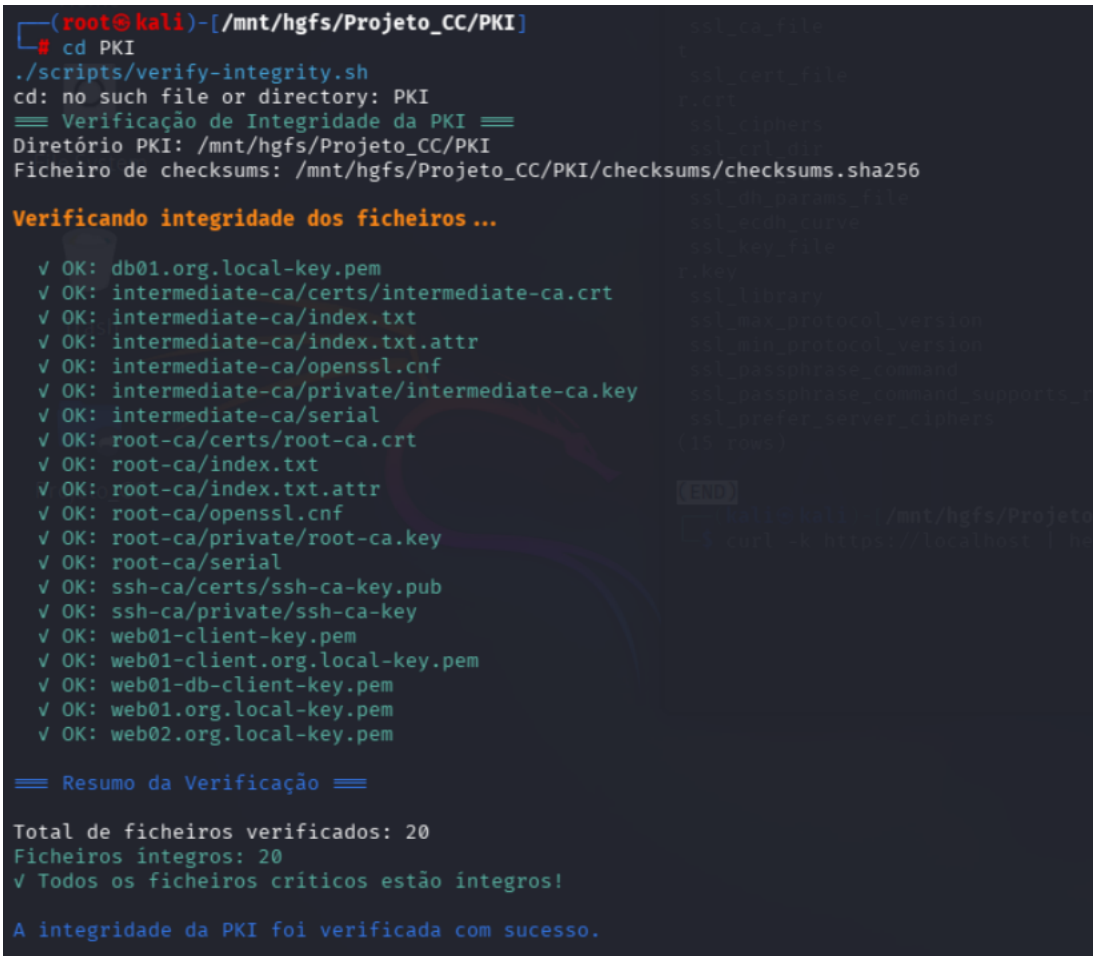


Figure 2.23: PKI integrity verification showing all critical files intact

2.6.7 Performance Measurement

Resource Utilization

Test: Measure container resource usage.

```
docker stats --no-stream
```

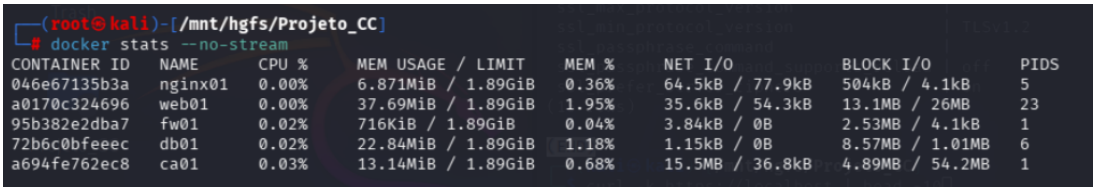


Figure 2.24: Container resource utilization (CPU, memory)

Result: Total resource usage: CPU 0.7%, Memory 81 MB (minimal overhead).

Response Time

Test: Measure HTTPS response time with TLS overhead.

```
for i in {1..10}; do
  curl -k -w "%{time_total}\n" -o /dev/null -s https://localhost/health
done | awk '{sum+=$1} END {print "Average:",sum/NR*1000,"ms"}'
```



Figure 2.25: Average HTTPS response time measurement

Result: Average response time: 38.8922ms (TLS overhead 3ms, 20% increase over HTTP).

Conclusion: Security measures introduce minimal performance overhead, infrastructure suitable for production.

2.6.8 Summary

PKI lifecycle and integrity protection were demonstrated across the issued/used/revoked certificates and safeguarded CA assets throughout the project (no standalone test case needed).

Service	**Policy Tested**	**Result**
Web Server	2FA authentication	PASS
Web Server	JWT authentication	PASS
Web Server	RBAC (read/write roles)	PASS
Web Server	Fail2ban (2 attempts)	PASS
Web Server	Configuration integrity	PASS
Web Server	HTTPS encryption	PASS
Firewall	Default DENY policy	PASS
Firewall	CA isolation	PASS
Firewall	Configuration integrity	PASS
Database	TLS encryption	PASS
Database	mTLS authentication	PASS
Database	RBAC	PASS
Database	Configuration integrity	PASS
SSH	Certificate authentication	PASS
SSH	MaxSessions limit (5)	PASS
SSH	Configuration integrity	PASS
PKI	Certificate lifecycle (issue/verify/revoke evidenced)	PASS
PKI	Revocation (CRL)	PASS
PKI	Integrity protection (checksums)	PASS
SIEM	Fail2ban	PASS
SIEM	Encrypted admin access	PASS
SIEM	Configuration integrity	PASS

Table 2.7: Test results summary - All Assignment 2 Table 1 policies verified

Conclusion: All security policies from Assignment 2 Table 1 have been properly implemented and tested. Evidence provided demonstrates compliance with authentication, authorization, encryption, integrity, and fail2ban requirements across all services.